

1 KNN

1.1 The Very Basics

AI Def:: the study of agents that receive percepts from the environment and perform actions. The goal is to design agents that “behave rationally”.

ML Def:: a subset of AI that enables systems to improve their performance on a task through experience; learns from and makes decisions based on data, without being explicitly programmed for each specific task.

DL Def:: a machine learning technique that uses multilayered neural networks to model complex patterns in data.

LLMs Def:: a class of deep learning models trained on vast amounts of text data to understand and generate human-like language.

Fundamental Ideas in ML: (1) Learning is optimization; (2) ML requires balancing tradeoffs in sources of error; (3) Model evaluation is empirical; (4) ML describes geometric processes; (5) ML demands a probabilistic lens.

Relation: AI (broad field of making machines behave intelligently) $\supset$ ML (learns patterns/rules from data instead of being explicitly programmed) $\supset$ DL (multi-layer neural networks to learn feature representations automatically).

Supervised Def:: learn a mapping $\vec{x} \rightarrow t$ from labelled input-target pairs.

Unsupervised Def:: only unlabelled data; discover structure (e.g. clustering, dimensionality reduction).

Reinforcement Def:: an agent interacts with an environment and learns a policy that maximizes cumulative reward; no correct answers given, only reward signals.

Learning: find a model f that maps from inputs to outputs given a labeled training dataset.

Inference: use a model f to predict an output y given a new example $\vec{x}$

A model is a function f mapping input $\vec{x}$ to output prediction y .

A model family is a set of models that share the same structure but differ in parameters (e.g. layers in a neural network)

Accuracy: accuracy = $\frac{1}{N} \sum_{i=1}^N \mathbb{I}[t^{(i)} = y^{(i)}]$.

Error rate: $1 - \text{accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[t^{(i)} \neq y^{(i)}]$.

A parameter is the internal variables of a model learned from data whilst training.

A hyperparameter are the settings chosen before training that controls how the learning process works.

Train: fit parameters.

Val: tune hyperparameters / pick model.

Test: When: use once after model & hyperparameters are chosen. Purpose: get an unbiased estimate of generalization error. If used for tuning: test info leaks into model selection, so the test error becomes optimistically biased. Regression vs classification is decided by the output only (continuous vs discrete).

t = target, y = prediction; $x_j^{(i)}$ = feature j of example i . 100% training accuracy $\nrightarrow$ good test accuracy; May overfit.

Data augmentation: changing the training data by a little bit to create new training data (shift/rotate).

Parametric models fix the parameter count. Non-parametric models don't. $O(1)$ space complexity with respect to the feature count, and never $\Omega(N)$. Hyperparameters are out of this scope.

1.2 KNN

1NN: predict $t^{(c)}$, where

$$\vec{x}^{(c)} = \arg \min_{\vec{x}^{(i)} \in \text{training set}} \text{distance}(\vec{x}^{(i)}, \vec{x})$$

kNN: predict using the majority class among the k closest training examples to $\mathbf{x}$.

Distances:

$$L_2: \sqrt{\sum_j (a_j - b_j)^2}$$

$$L_\infty: (\sum_j |a_j - b_j|^p)^{1/p}$$

Cos: ignores vector length, only direction $\rightarrow$ good for text.

Normalize: features to mean = 0 and variance = 1,

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Rule: compute μ_j, σ_j from the training set only; Reuse them for validation/test (else leakage).

Boundary (1NN):

Euclidean: (1) for nearby pairs of different classes draw the perpendicular bisector ($\mathbf{b} - \mathbf{a}$) $^T \mathbf{x} = \frac{\|\mathbf{b}\|^2 - \|\mathbf{a}\|^2}{2}$; (2) cut bisectors at Voronoi vertices (points equidistant to 3 training points); (3) keep only edges separating different classes; (4) check with a test point.

Manhattan: bisector of 2 points depends on their bounding box: wide ($|\Delta x_1| > |\Delta x_2|$) $\rightarrow$ 45° segment inside the box, then vertical rays; Tall $\rightarrow$ 45° segment, then horizontal rays; Square $\rightarrow$ diagonal + 2-D tie regions (state the tie-breaking rule).

Cosine: only the angle matters $\rightarrow$ regions are wedges from the origin; Boundary = rays along angle bisectors of adjacent points of different classes.

Classify a point: Steps: list d^2 to every training point (skip $\surd$, same order) $\rightarrow$ take the k smallest $\rightarrow$ majority vote; Tie $\rightarrow$ use the stated tie rule. 1NN says A, kNN says B: overlay both boundaries, pick a point strictly inside the overlap (on a boundary = tie).

Training accuracy (kNN): Formula: $\frac{1}{N_{\text{train}}} \sum_i \mathbb{I}[y(\mathbf{x}^{(i)}) = t^{(i)}]$. Rule: each training point is its own nearest neighbour ($d = 0$) $\rightarrow$ its vote is its own label + its $k-1$ nearest other points. Why 1NN = 100%: the only neighbour is the point itself $\rightarrow$ always predicts its own label (unless duplicate $\mathbf{x}$ with different labels). Why 3NN not: its own label is only 1 of 3 votes; The 2 nearest other points can outvote it if both have a different label.

Pros: simple; No training; Works for regression and classification; Adapts easily to new data.

Cons:

(1) **Curse of dimensionality:** in high dimensions most points are far apart from each other (L_∞ on $[0, 1]^d$: fraction within ϵ of origin = ϵ^d ; $\therefore \epsilon < 1, d \uparrow \Rightarrow \epsilon^d \rightarrow 0$) $\rightarrow$

Sol: add more data or reduce dimensionality.

(2) Sensitive to ranges of features (larger range dominates distance calculations) $\rightarrow$ *Sol:* normalization.

(3) Slow prediction: distance to all N points, $O(ND)$ per query.

(4) Must store the entire training set, $O(ND)$ space.

(5) Discrete features (e.g. Outlook $\in$ {Sunny, Overcast, Rain}): values aren't numeric $\rightarrow$ no clear distance; Prefer DT.

Choose k :

$k \uparrow \rightarrow$ boundary smoother, less sensitive to individual points $\rightarrow$ underfit; $k = N \rightarrow$ always predicts the majority class.

$k = 1 \rightarrow$ sharp boundary, overfit; 100% training accuracy (unless duplicate $\mathbf{x}$ with different labels).

No single best k , depends on N ; Heuristic $k < \sqrt{N}$ ($k \rightarrow \infty, k/N \rightarrow 0$).

Pick k (any hyperparameter) via validation set or cross-validation.

Workflow:

(1) Split data into train/validation/test.

(2) For each K : fit KNN on train.

(3) Compute accuracy of each on validation.

(4) Pick the K with the best validation accuracy.

(5) Report the test accuracy of that K .

2 Decision Trees

Tree size: large $\rightarrow$ overfit; Small $\rightarrow$ underfit.

Inference: trace the tree from root to leaf.

Entropy:

$$H(Y) = -\mathbb{E}_{Y \sim p(y)} [\log_2 p(Y)] = -\sum_y p(y) \log_2 p(y)$$

Notation $H(p_1, \dots, p_n)$: Def: entropy of a distribution whose probabilities are $p_1, \dots, p_n$ (arguments are numbers, $\sum_k p_k = 1$): $H(p_1, \dots, p_n) = -\sum_k p_k \log_2 p_k$.

Property: $H(\frac{1}{2}, \frac{1}{2}) = -2 \cdot \frac{1}{2} \log_2 \frac{1}{2} = 1$ (max for 2 classes; 50/50 split = most uncertain).

Joint entropy: Def: uncertainty of the pair (X, Y) (arguments are random variables, not probabilities):

$$\begin{aligned} H(X, Y) &= -\mathbb{E}_{X, Y \sim p(x, y)} [\log_2 p(X, Y)] \\ &= -\sum_{x, y} p(x, y) \log_2 p(x, y) \end{aligned}$$

Chain rule: $H(X, Y) = H(X) + H(Y | X)$; If independent:

$H(X, Y) = H(X) + H(Y)$.

Conditional entropy:

$$\begin{aligned} H(Y | X = x) &= -\mathbb{E}_{Y \sim p(y|x)} [\log_2 p(Y | x)] \\ &= -\sum_y p(y | x) \log_2 p(y | x) \end{aligned}$$

Expected conditional entropy:

$$\begin{aligned} H(Y | X) &= \mathbb{E}_{X \sim p(x)} [H(Y | X = x)] \\ &= \sum_x p(x) H(Y | X = x) \\ &= -\mathbb{E}_{X, Y \sim p(x, y)} [\log_2 p(Y | X)] \\ &= -\sum_x \sum_y p(x, y) \log_2 p(y | x) \end{aligned}$$

Information gain:

$$IG(Y | X) = H(Y) - H(Y | X)$$

$0 \leq IG(Y | X) \leq H(Y)$, i.e. $IG(Y | X) \in [0, H(Y)]$.

IG table (Y = label/class, X = split on a feature; count examples per split side & class):

	Class 1	Class 2	Total
Split cond. 1	a	b	a+b
Split cond. 2	d	c	c+d
Total	a+c	b+d	N

$$H(Y) = H\left(\frac{a+b}{N}, \frac{b+d}{N}\right)$$

$$H(Y | X) = \frac{a+b}{N} H\left(\frac{a}{a+b}, \frac{b}{a+b}\right) + \frac{c+d}{N} H\left(\frac{c}{c+d}, \frac{d}{c+d}\right)$$

Properties:

$H(Y) \geq 0$; Certain outcome $\rightarrow H = 0$; Binary: max H is 1 bit at $p = 0.5$.

$H(Y | X) \leq H(Y)$ (i.e. $IG \geq 0$; $\therefore$ observing X can't increase uncertainty about Y).

Y, X independent $\Rightarrow IG(Y | X) = 0$ (i.e. $H(Y | X) = H(Y)$).

Y fully determined by $X \Rightarrow H(Y | X) = 0 \Rightarrow IG(Y | X) = H(Y)$ (i.e. max $IG(Y | X)$).

2.1 Decision Tree Algorithm

Stop splitting when:

(1) All examples have the same label $\rightarrow$ leaf = that label.

(2) No features or split points left $\rightarrow$ leaf = majority class.

When: examples with identical feature values but different labels; A real-valued feature has no split points once all its values in the node are equal.

(3) No examples left $\rightarrow$ leaf = parent's majority class.

When: a value of a multi-way split has no examples; Never happens if every feature-value combination appears in the data.

(4) Max depth reached (hyperparameter).

(5) Loss decreases too little (hyperparameter).

(6) Node has fewer than a min number of samples (hyperparameter).

Split: pick the split with max IG (not training error, $\therefore$ it ignores the uncertainty left in the child nodes).

Leaf prediction: Class: majority vote; Reg: mean.

Properties:

Greedy $\rightarrow$ not globally optimal ($\therefore$ finding the optimal tree is NP-hard).

Boundaries are axis-aligned (each split tests $x_j \leq \theta$).

Continuous feature: only $\leq N-1$ thresholds to check (mid-points between sorted values).

Scale-invariant.

Boundary:

(1) Each split $x_j \leq \theta$ is a vertical/horizontal line, drawn only inside its parent node's region.

(2) Leaves = axis-aligned rectangles.

(3) Label each rectangle with its leaf's prediction (majority class).

Train cost vs KNN: DT more ($\therefore$ compute IG of each split to choose the best feature & threshold); KNN less (no computation, just stores data).

Pros (vs KNN): handles discrete features directly (one branch per value, no distance needed).

3 Linear / GD

Setup: N = number of examples, D = number of input features.

Dummy: $x_0^{(i)} = 1 \Rightarrow b = w_0 x_0^{(i)}$:

$$\mathbf{x}^{(i)} = [1, x_1^{(i)}, \dots, x_D^{(i)}]^T \in \mathbb{R}^{(D+1) \times 1}$$

$$\mathbf{w} = [w_0, \dots, w_D]^T \in \mathbb{R}^{(D+1) \times 1}$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} (= \mathbf{x}^{(i)T} \mathbf{w}) = \sum_{j=0}^D x_j^{(i)} w_j \in \mathbb{R}$$

$$X = \begin{bmatrix} (\mathbf{x}^{(1)})^T \\ \vdots \\ (\mathbf{x}^{(N)})^T \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \text{ (row = example)}$$

$$\mathbf{t}, \mathbf{y} \in \mathbb{R}^{N \times 1}, \mathbf{y} = \mathbf{X} \mathbf{w}$$

Prediction: $y = \mathbf{w}^T \mathbf{x} + b, \mathbf{y} = \mathbf{X} \mathbf{w}$ ($X \in \mathbb{R}^{N \times (D+1)}$, row = example).

Gradient direction: ∇ = direction of steepest ascent, perpendicular to contours $\rightarrow$ GD steps along $-\nabla$.

Feature map: $\psi(\mathbf{x})$ (e.g. $[1, x, x^2]$) $\rightarrow$ fits nonlinear functions of $\mathbf{x}$, model still linear in $\mathbf{w}$.

Train/val error vs complexity: Train: $\downarrow$ as complexity $\uparrow$; Val: $\downarrow$ then $\uparrow$ (U-shape).

Underfit: low accuracy: both high, small gap, both still $\downarrow \rightarrow$ increase complexity.

Overfit: high complexity: train $\downarrow$ (to 0), val $\uparrow$, gap large & growing $\rightarrow$ fits noise.

Good fit: val at its min, moderate gap $\rightarrow$ best bias-variance trade-off; Choose this complexity.

$$\text{Cost function: } \mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$$

$$\text{Loss function: } \mathcal{L}(y, t) = \frac{1}{2} (y - t)^2$$

Loss = how off one single example is; Cost = average over a set of examples.

Linear cost:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)})^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (\mathbf{X} \mathbf{w} - \mathbf{t})^T (\mathbf{X} \mathbf{w} - \mathbf{t})$$

3.1 Gradient of the Cost Function

$$\mathcal{L}(y^{(i)}, t^{(i)}) = \frac{1}{2} (y^{(i)} - t^{(i)})^2 \in \mathbb{R}$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_i \mathcal{L} \in \mathbb{R}$$

$$\text{Let } \mathbf{e} = \mathbf{y} - \mathbf{t} \in \mathbb{R}^{N \times 1}: \mathbf{e}^T \mathbf{e} = \sum_i (y^{(i)} - t^{(i)})^2$$

$$\begin{aligned} \Rightarrow \mathcal{E} &= \frac{1}{2N} \mathbf{e}^T \mathbf{e} = \frac{1}{2N} (\mathbf{X} \mathbf{w} - \mathbf{t})^T (\mathbf{X} \mathbf{w} - \mathbf{t}) \\ &= \frac{1}{2N} \|\mathbf{X} \mathbf{w} - \mathbf{t}\|_2^2 \end{aligned}$$

Gradient:

$$\frac{\partial \mathcal{L}}{\partial y^{(i)}} = y^{(i)} - t^{(i)}, \quad \frac{\partial y^{(i)}}{\partial w_j} = x_j^{(i)}$$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial w_j} = (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\Rightarrow \frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\Rightarrow \nabla_{\mathbf{w}} \mathcal{L} = (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)} \in \mathbb{R}^{D+1}$$

$$\Rightarrow \nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

$$\text{Let } \mathbf{r}^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)} \in \mathbb{R}, \mathbf{r} = \mathbf{X} \mathbf{w} - \mathbf{t} \in \mathbb{R}^{N \times 1}:$$

$$\begin{aligned} \nabla_{\mathbf{w}} \mathcal{E} &= \frac{1}{N} \sum_i r^{(i)} \mathbf{x}^{(i)} = \frac{1}{N} \mathbf{X}^T \mathbf{r} \\ &= \frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{t}) \in \mathbb{R}^{D+1} \end{aligned}$$

KEY: $\sum_i r^{(i)} \mathbf{x}^{(i)} = \mathbf{X}^T \mathbf{r}$

($\therefore$ columns of X^T are $\mathbf{x}^{(i)}$, weighted by $r^{(i)}$)

Solve:

$$(1) \text{ Direct (closed form): } \nabla \mathcal{E} = 0 \Rightarrow X^T X \mathbf{w} = X^T \mathbf{t}$$

$$\Rightarrow \mathbf{w} = (X^T X)^{-1} X^T \mathbf{t}$$

Pros: closed form, exact & provably optimal, no iteration.

Cons: closed-form solution may not exist for most problems; Can't generalize to most models; Computationally expensive: $(X^T X)^{-1}$ is $O(D^3)$.

(2) **GD update:**

$$\text{Non-vectorized: } w_j \leftarrow w_j - \frac{\alpha}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\text{Vectorized: } \underbrace{\mathbf{w}}_{\in \mathbb{R}^{(D+1) \times 1}} \leftarrow \mathbf{w} - \underbrace{\frac{\alpha}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{t})}_{\in \mathbb{R}^{(D+1) \times 1}}$$

Learning rate α : too big $\rightarrow$ overshoot/diverge; Too small $\rightarrow$ converges slowly.

Large Weight: Why problematic: one feature contributes a lot to the prediction, but a good prediction uses many features $\rightarrow$ sign of overfitting.

Regularization:

Def: modify training to choose models using criteria beyond training error, i.e. minimize $\mathcal{E} + \lambda \mathcal{R}$ (penalize large $\mathbf{w}$).

Effect of λ : small $\rightarrow$ overfit; Big $\rightarrow$ underfit.

Not: forcing weights to similar scale; Not guaranteeing 0 training error; Adds (not removes) hyperparameter λ .

$$\text{L2 regularization: } \mathcal{R} = \frac{1}{2} \sum_{j=0}^D w_j^2$$

∇ regularizer:

$$\begin{aligned} \frac{\partial}{\partial w_j} \left(\frac{\lambda}{2} \sum_{j=0}^D w_j^2 \right) &= \lambda w_j \\ \nabla_{\mathbf{w}} \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) &= \lambda \mathbf{w} \end{aligned}$$

Gradient (regularized):

$$\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{t}) + \lambda \mathbf{w} \in \mathbb{R}^{D+1}$$

GD update (regularized)

Non-vectorized: $w_j \leftarrow w_j - \alpha (\frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)} + \lambda w_j)$

$$\text{Vectorized: } \mathbf{w} \leftarrow \mathbf{w} - \alpha (\frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{t}) + \lambda \mathbf{w})$$

Direct (regularized): Steps: set $\nabla_{\mathbf{w}} \mathcal{E} = 0$, multiply by N , write $\mathbf{w} = I \mathbf{w}$ ($I \in \mathbb{R}^{(D+1) \times (D+1)}$):

$$\begin{aligned} X^T X \mathbf{w} + N \lambda \mathbf{w} &= X^T \mathbf{t} \\ (X^T X + N \lambda I) \mathbf{w} &= X^T \mathbf{t} \\ \mathbf{w} &= (X^T X + N \lambda I)^{-1} X^T \mathbf{t} \end{aligned}$$

L1 regularization: $\mathcal{R} = \sum_{j=0}^D |w_j|$
Effect: drives many w_j exactly to $0 \rightarrow$ sparse $\mathbf{w}$ (feature selection); L2 only shrinks w_j toward 0.
 ∇ regularizer: $\frac{\partial}{\partial w_j}(\lambda \sum_j |w_j|) = \lambda \operatorname{sgn}(w_j)$ (not differentiable at $w_j = 0$); $\nabla_{\mathbf{w}} = \lambda \operatorname{sgn}(\mathbf{w})$
Gradient (L1):
 $\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)} + \lambda \operatorname{sgn}(w_j)$
 $\nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} X^T (X\mathbf{w} - \mathbf{t}) + \lambda \operatorname{sgn}(\mathbf{w}) \in \mathbb{R}^{D+1}$
GD update (L1):
Non-vectorized: $w_j \leftarrow w_j - \alpha (\frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)} + \lambda \operatorname{sgn}(w_j))$
Vectorized: $\mathbf{w} \leftarrow \mathbf{w} - \alpha (\frac{1}{N} X^T (X\mathbf{w} - \mathbf{t}) + \lambda \operatorname{sgn}(\mathbf{w}))$
Direct: no closed form.
With bias term:

$$\begin{aligned}\mathcal{E}(\mathbf{w}) &= \frac{1}{2N} (X\mathbf{w} + b\mathbf{1} - \mathbf{t})^T (X\mathbf{w} + b\mathbf{1} - \mathbf{t}) \\ \nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) &= \frac{1}{N} X^T (X\mathbf{w} + b\mathbf{1} - \mathbf{t}) \\ \frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_j} &= \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)}) x_j^{(i)} \\ \frac{\partial \mathcal{E}(\mathbf{w})}{\partial b} &= \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)}) \\ &= \frac{1}{N} \mathbf{1}^T (X\mathbf{w} + b\mathbf{1} - \mathbf{t})\end{aligned}$$

GD update (with bias):
Non-vectorized: $w_j \leftarrow w_j - \frac{\alpha}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)}) x_j^{(i)}$,
 $b \leftarrow b - \frac{\alpha}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)})$
Vectorized: $\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} X^T (X\mathbf{w} + b\mathbf{1} - \mathbf{t})$,
 $b \leftarrow b - \frac{\alpha}{N} \mathbf{1}^T (X\mathbf{w} + b\mathbf{1} - \mathbf{t})$
Direct (with bias, 1 feature, $y = wx + b$): *Steps:* set $\frac{\partial \mathcal{E}}{\partial w} = 0$ and $\frac{\partial \mathcal{E}}{\partial b} = 0 \rightarrow 2$ linear equations in $w, b \rightarrow$ plug in the data points:
 $\frac{1}{N} \sum_i (wx^{(i)} + b - t^{(i)}) x^{(i)} = 0$, $\frac{1}{N} \sum_i (wx^{(i)} + b - t^{(i)}) = 0$
Normal eq for 1 feat: *Equations:* multiply the 2 Direct (with bias) equations by N , expand ($\sum = \sum_{i=1}^N$):
 $w \sum (x^{(i)})^2 + b \sum x^{(i)} = \sum x^{(i)} t^{(i)}$
 $w \sum x^{(i)} + bN = \sum t^{(i)}$

Matrix form: same as $X^T X \mathbf{w} = X^T \mathbf{t}$ with row i of $X = [x^{(i)}, 1]$:

$$\begin{bmatrix} \sum x^{(i)} x^{(i)} & \sum x^{(i)} \\ \sum x^{(i)} & N \end{bmatrix} \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} \sum x^{(i)} t^{(i)} \\ \sum t^{(i)} \end{bmatrix}$$

Solution:

$$w = \frac{N \sum x^{(i)} t^{(i)} - \sum x^{(i)} \sum t^{(i)}}{N \sum (x^{(i)})^2 - (\sum x^{(i)})^2}$$

$$b = \frac{\sum t^{(i)} - w \sum x^{(i)}}{N} = \bar{t} - w \bar{x}$$

($\bar{x}, \bar{t}$ = means of $x^{(i)}, t^{(i)}$)

3.2 Stochastic GD

SGD update (i random, 1 example $\Rightarrow$ no $\frac{1}{N} \sum_i$): $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}^{(i)}$; For LinReg:
Non-vectorized: $w_j \leftarrow w_j - \alpha (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$
Vectorized: $\mathbf{w} \leftarrow \mathbf{w} - \alpha (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$
Iteration Def:: a single update step in the optimization algorithm. In SGD, one iteration = processing one mini-batch and updating the parameters once.
Epoch Def:: one complete pass through the entire training dataset. In SGD, one epoch = processing all mini-batches exactly once.

	Batch	Mini-batch	SGD
#examples/update	all N	$B, 1 < B < N$	1
Gradient accuracy	exact	noisy ($\downarrow$ as $B \uparrow$)	noisiest
Cost/update	grows with N	grows with B	1 example
Vectorization	High, but the full dataset may not fit in memory	High, and B can be chosen to fit the hardware	Low, one example at a time

Mini-batch gradient: unbiased estimate of the full gradient.
SGD Pros: noisy gradient $\rightarrow$ can escape flat regions (e.g. plateaus); No guarantee of reaching the global optimum.
Ravine:
Def: cost is steep in some dimensions, flat in others (gradient magnitudes differ a lot across dimensions).
Effect: GD oscillates in steep dimensions, moves slowly in flat dimensions.
Cause: features on very different scales.
Fix: normalize features.

3.3 Vectorizing Tips

$X \in \mathbb{R}^{N \times D}$; Other variables' shapes are inferred.
Answer format: vectors are columns; For every new symbol state its dimension and define its components (e.g. $X \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{x}_i^T$).
Dot product:

$$\sum_{i=1}^N u_i v_i = \mathbf{u}^T \mathbf{v} = \mathbf{v}^T \mathbf{u}$$

Hadamard product:

$$\mathbf{u} \odot \mathbf{v} = \operatorname{diag}(\mathbf{u}) \mathbf{v} = \left(\mathbf{u}^T \operatorname{diag}(\mathbf{v}) \right)^T$$

Sum a vector:

$$\mathbf{u}^T \mathbf{1} = \sum_{i=1}^N u_i$$

Sum axis 1: horizontal squish $> - <$ ($X1 \in \mathbb{R}^{N \times 1}$)

$$X1$$

Sum axis 0: vertical squish $\downarrow$ ($1^T X \in \mathbb{R}^{1 \times D}$)

$$1^T X$$

May need to transpose.
Weighted sum axis 1: weighted horizontal squish ($M \in \mathbb{R}^{P \times Q}$, column $j = \mathbf{m}_j \in \mathbb{R}^{P \times 1}$, weights $\mathbf{u} \in \mathbb{R}^{Q \times 1}$)

$$\sum_{j=1}^Q u_j \mathbf{m}_j = M \mathbf{u} \in \mathbb{R}^{P \times 1}$$

Weighted sum axis 0: weighted vertical squish ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$, $\mathbf{m}_i \in \mathbb{R}^{Q \times 1}$, weights $\mathbf{u} \in \mathbb{R}^{P \times 1}$)

$$\sum_{i=1}^P u_i \mathbf{m}_i^T = \mathbf{u}^T M \in \mathbb{R}^{1 \times Q}$$

Transpose of above (column form):

$$\sum_{i=1}^P u_i \mathbf{m}_i = M^T \mathbf{u} \in \mathbb{R}^{Q \times 1}$$

Row-wise multiplication ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$, $\mathbf{m}_i \in \mathbb{R}^{Q \times 1}$; $\mathbf{u} \in \mathbb{R}^{P \times 1}$):

$$\operatorname{diag}(\mathbf{u}) M = \begin{bmatrix} - & u_1 \mathbf{m}_1^T & - \\ - & u_i \mathbf{m}_i^T & - \\ - & u_P \mathbf{m}_P^T & - \end{bmatrix} \in \mathbb{R}^{P \times Q}$$

Column-wise multiplication ($M \in \mathbb{R}^{P \times Q}$, column $j = \mathbf{m}_j \in \mathbb{R}^{P \times 1}$; $\mathbf{v} \in \mathbb{R}^{Q \times 1}$):

$$M \operatorname{diag}(\mathbf{v}) = \begin{bmatrix} v_1 \mathbf{m}_1 & v_j \mathbf{m}_j & v_Q \mathbf{m}_Q \\ \vdots & \vdots & \vdots \end{bmatrix} \in \mathbb{R}^{P \times Q}$$

Shape reasoning: the summed index (N) must be the *inner* dimension so it cancels; The first dimension of the result goes on the left.
Scalar with vector ($s \in \mathbb{R}$):

$$\begin{aligned}\alpha_i &= u_i + s \Rightarrow \boldsymbol{\alpha} = \mathbf{u} + s \mathbf{1} \\ \beta_i &= s u_i \Rightarrow \boldsymbol{\beta} = s \mathbf{u}\end{aligned}$$

Outer product:

$$M_{ij} = u_i v_j \Rightarrow M = \mathbf{u} \mathbf{v}^T \in \mathbb{R}^{P \times Q}$$

Output type: inner product $\rightarrow$ scalar; Element-wise $\rightarrow$ vector; Outer $\rightarrow$ matrix.
Stack of dot products ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$; $\mathbf{u} \in \mathbb{R}^{Q \times 1}$):

$$\alpha_i = \mathbf{m}_i^T \mathbf{u} \Rightarrow \boldsymbol{\alpha} = M \mathbf{u} \in \mathbb{R}^{P \times 1}$$

Two weights per term ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$; $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{P \times 1}$):

$$\sum_i u_i v_i \mathbf{m}_i = M^T (\mathbf{u} \odot \mathbf{v}) = M^T \operatorname{diag}(\mathbf{u}) \mathbf{v} \in \mathbb{R}^{Q \times 1}$$

Double sums:

$$\begin{aligned}\sum_i \sum_j u_i v_j m_{ij} &= \mathbf{u}^T M \mathbf{v} \\ \sum_i \sum_j u_{ij} v_{m_{ij}} &= \mathbf{u}^T (M \odot M) \mathbf{v} \\ \sum_i \sum_k u_{ik} m_{ik} u_k &= \mathbf{u}^T M \mathbf{u}\end{aligned}$$

Pitfall: $\mathbf{a} \odot \mathbf{b} \neq \mathbf{a} \operatorname{diag}(\mathbf{b}) - \mathbf{a} \in \mathbb{R}^{P \times 1}$ times $\operatorname{diag}(\mathbf{b}) \in \mathbb{R}^{P \times P}$ is undefined; Use $\operatorname{diag}(\mathbf{b}) \mathbf{a}$.
Pitfall: $\frac{1}{N} (X\mathbf{w} - \mathbf{t})^T X \in \mathbb{R}^{1 \times (D+1)}$ is a *row*; Gradient must be a column $\Rightarrow \frac{1}{N} X^T (X\mathbf{w} - \mathbf{t})$.
Variants of linear regression gradient ($\mathbf{r} = X\mathbf{w} - \mathbf{t}$):

- Per-weight λ_j : $\frac{1}{N} X^T \mathbf{r} + \lambda \odot \mathbf{w} = \frac{1}{N} X^T \mathbf{r} + \operatorname{diag}(\lambda) \mathbf{w}$
- Per-example weight α_i : $\sum_i \alpha_i \mathbf{r}^{(i)} \mathbf{x}^{(i)} = X^T \operatorname{diag}(\boldsymbol{\alpha}) \mathbf{r}$
- Per-feature scale α_j : predictions $X \operatorname{diag}(\boldsymbol{\alpha}) \mathbf{w}$, gradient gains a $\operatorname{diag}(\boldsymbol{\alpha})$ on the left: $\frac{1}{N} \operatorname{diag}(\boldsymbol{\alpha}) X^T \mathbf{r}'$, $\mathbf{r}' = X \operatorname{diag}(\boldsymbol{\alpha}) \mathbf{w} - \mathbf{t}$

Absorb bias: $\mathbf{w}_1^T \mathbf{x}_1 + b = \mathbf{w}_2^T \mathbf{x}_2$ with $\mathbf{w}_2 = \begin{bmatrix} b \\ \mathbf{w}_1 \end{bmatrix}$,
 $\mathbf{x}_2 = \begin{bmatrix} 1 \\ \mathbf{x}_1 \end{bmatrix}$ ($\mathbf{w}_2, \mathbf{x}_2 \in \mathbb{R}^{(D+1) \times 1}$).
Quadratic form gradient:

$$\nabla_{\mathbf{w}} \|A\mathbf{w} - \mathbf{t}\|^2 = 2A^T (A\mathbf{w} - \mathbf{t})$$

4 Logistic Regression

Output: $y = \sigma(\mathbf{w}^T \mathbf{x}) = p(t = 1 \mid \mathbf{x})$, threshold 0.5.
Linear model:

$$z = \mathbf{w}^T \mathbf{x}$$

Prediction:

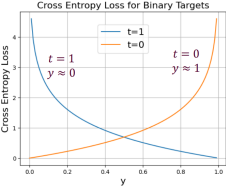
$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Loss functions:

- Zero-One Loss:** $L_{0-1}(y, t) = \mathbb{I}[y \neq t]$
- Non-differentiable / zero gradient $\rightarrow$ can't update $\mathbf{w}$.
- Squared Error Loss:** $L_{SE}(z, t) = (\frac{1}{2} (z - t)^2)$
- Penalizes confident correct predictions.
- Sigmoid + Squared Error:**
 $y = \mathbb{I}(z) = \frac{1}{1+e^{-z}}$, $L_{SE}(y, t) = \frac{1}{2} (y - t)^2$
- When the prediction is very wrong $\rightarrow \sigma$ saturates $\rightarrow$ the gradient is close to 0.
- Step + Squared Error:**
 $y = \mathbb{I}[z \geq 0.5]$, $L_{SE}(y, t) = \frac{1}{2} (y - t)^2$
- $\frac{\partial y}{\partial z} = 0$ almost everywhere $\rightarrow$ zero gradient, can't update $\mathbf{w}$ (same as 0-1 loss).

• **Cross entropy:** $\begin{cases} -\log(y) & t = 1 \\ -\log(1 - y) & t = 0 \end{cases}$

Cross entropy:

$$-t \log(y) - (1 - t) \log(1 - y)$$


4.1 Cross-Entropy Loss Equations

Linear model:

$$z = \mathbf{w}^T \mathbf{x}$$

Classification:

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Loss:

$$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1 - t) \log(1 - y)$$

Gradient:
Where $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$; for a single example, dropping (i):

$$\begin{aligned}\frac{\partial \mathcal{L}_{CE}}{\partial w_j} &= \frac{\partial \mathcal{L}_{CE}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} \\ &= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j \\ &= \frac{y-t}{y(1-y)} \cdot y(1-y) \cdot x_j \\ &= (y-t)x_j\end{aligned}$$

$\Rightarrow \frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)}$
 $\Rightarrow \nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} X^T (\mathbf{y} - \mathbf{t})$, $\mathbf{y} = \sigma(X\mathbf{w}) \in \mathbb{R}^{N \times 1}$
GD update:
Non-vectorized: $w_j \leftarrow w_j - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)}$
Vectorized: $\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} X^T (\sigma(X\mathbf{w}) - \mathbf{t})$
Pros:
 $\mathcal{L} \rightarrow \infty$ for confident wrong predictions.
 $\partial \mathcal{L} / \partial z = y - t$ (no saturation).
Convex in $\mathbf{w}$; Maximum likelihood.
Sigmoid+SE Cons: $\partial \mathcal{L} / \partial z = (y - t)y(1 - y) \approx 0$ when the prediction is very wrong.
Boundary (threshold a : predict 1 iff $y \geq a$):

$$\begin{aligned}\sigma(z) = a &\iff 1 + e^{-z} = \frac{1}{a} \iff e^{-z} = \frac{1-a}{a} \\ &\iff z = \mathbf{w}^T \mathbf{x} = \ln \frac{a}{1-a}\end{aligned}$$

Hyperplane; Class 1 where $z \geq \ln \frac{a}{1-a}$ ($\cdot$: σ increasing).
If $a = 0.5$: $\ln \frac{0.5}{0.5} = \ln 1 = 0 \Rightarrow$ boundary $\mathbf{w}^T \mathbf{x} = 0$; Class 1 where $z \geq 0$.
Steps:
(1) Write $z = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2$ (with $x_0 = 1$).
(2) Set $z = \ln \frac{a}{1-a}$ ($= 0$ if $a = 0.5$) and solve for $x_2 \rightarrow$ boundary line.
(3) Plug a test point (e.g. origin) into z : the side with $z > \ln \frac{a}{1-a}$ is class 1, the other side is class 0.
(4) Draw the line through two easy points (intercepts) and label both regions.

4.2 LogReg vs kNN

Parametric: hypothesis space characterized by a fixed-size set of parameters (e.g. LogReg).
Non-parametric: complexity can grow with the amount of training data (e.g. kNN).
Pros (vs kNN):
Storage: only $D+1$ weights, training data discarded after fitting (parametric).
Prediction: fast, one dot product $\mathbf{w}^T \mathbf{x}$ with fixed learned weights, $O(D)$.
Interpretability: weights rank the features.
Generalization: good from limited data if the linear assumption holds.
Cons:
Inductive bias: linear decision boundary $\rightarrow$ underfits a nonlinear boundary.
Fixed number of parameters $\rightarrow$ complexity doesn't grow with more data (unlike kNN).

	LogReg	kNN
Model type	Parametric	Non-parametric
Inductive bias	Linear decision boundary	Nearby inputs share labels
#Parameters	$D+1$ weights	Grows with the data
Storage	$D+1$ weights	Entire training set
Prediction cost	One dot product	Distance to every example
Interpretability	Weights rank the features	Neighbours explain one prediction
Typical failure	Underfits a nonlinear boundary	Overfits when k is small

4.3 Softmax Regression ($K \geq 3$)

Target: one-hot $\mathbf{t} \in \{0, 1\}^K$ (1 at true class). *Why not integers:* imply order + distance, renumbering changes model.
Why one-hot: no order, all pairs equidistant.
Logits: $z = W\mathbf{x} \in \mathbb{R}^K$, $W \in \mathbb{R}^{K \times (D+1)}$, row $k = [b_k, \mathbf{w}_k^T]$,
 $z_k = \mathbf{w}_k^T \mathbf{x}$ (unnormalized). Predict $\arg \max_k z_k$.

$y_k = \frac{\exp(z_k)}{\sum_{k'=1}^K \exp(z_{k'})}$, $\mathbf{y} = \operatorname{softmax}(\mathbf{z}) \in \mathbb{R}^K$
Properties:
(1) Probability distribution: $y_k \in (0, 1)$, $\sum_k y_k = 1$.
(2) Order-preserving ($\cdot$: exp $\uparrow$) $\Rightarrow \arg \max y = \arg \max z$, softmax doesn't change the boundary; $y_k = \text{confidence}$.
(3) Smooth, differentiable approximation of $\arg \max \Rightarrow$ GD works.
(4) $K = 2$: $y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2)$, $y_2 = \sigma(-z)$.
Loss: $\mathcal{L}_{CE} = -\sum_k t_k \log y_k = -\mathbf{t}^T \log \mathbf{y} = -\log y_{\text{true}}$ (1 nonzero term).
Gradient:

$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} = y_k - t_k \Rightarrow \nabla_{\mathbf{w}_k} \mathcal{L} = (y_k - t_k) \mathbf{x}$
 $\Rightarrow \nabla_{\mathbf{w}_k} \mathcal{E} = \frac{1}{N} \sum_{i=1}^N (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$
 $\Rightarrow \nabla_W \mathcal{E} = \frac{1}{N} (Y - T)^T X \in \mathbb{R}^{K \times (D+1)}$ ($Y, T \in \mathbb{R}^{N \times K}$, rows $\mathbf{y}^{(i)T}, \mathbf{t}^{(i)T}$)
GD update:
Non-vectorized: $\mathbf{w}_k \leftarrow \mathbf{w}_k - \frac{\alpha}{N} \sum_{i=1}^N (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$ (each k)
Vectorized: $W \leftarrow W - \frac{\alpha}{N} (Y - T)^T X$
Boundary i/j : needs BOTH, (1) $z_i = z_j \iff (\mathbf{w}_i - \mathbf{w}_j)^T \mathbf{x} = 0$ (line); (2) $z_k \leq z_i \forall k \neq i, j$.
Steps: solve (1) for line $\rightarrow$ substitute into (2) $\rightarrow$ inequality gives ray/segment $\rightarrow$ all 3 lines meet where $z_1 = z_2 = z_3 \rightarrow$ test a point per region. At most $K(K-1)/2$ boundaries.
Cons: linear boundaries only (convex regions); Can't do nonlinear.